CS 486/686
Sequence Models, Attention, and Transformers

Yuntian Deng

Lecture 19

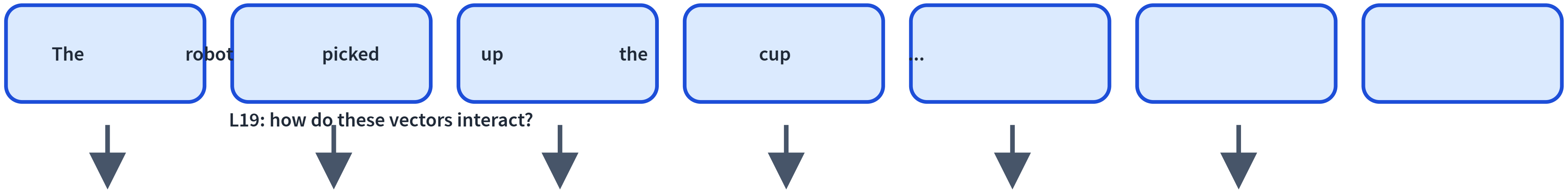
How embeddings talk to each other

Learning goals

- Model a sentence as a **sequence of embeddings**.
- Explain **RNNs** and why they struggle to scale.
- Explain **attention** as token-to-token retrieval.
- Assemble a **transformer block** and understand **causal masking**.

From embeddings to sequences

L18: each item can become a learned vector.



A sentence is not a bag of words

dog bites man	man bites dog

Same words. Different order. Different meaning.

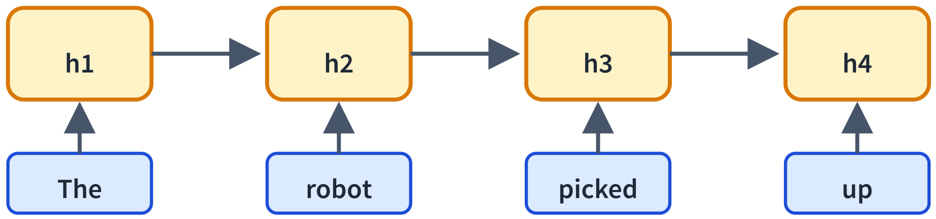
Running example

The robot picked up the cup because **it** was empty.

What does **it** refer to?

The answer depends on context, not the token alone.

The first idea: recurrent state



Read one token at a time.

$$h_t = f(h_{t-1}, x_t)$$

The hidden state summarizes the past.

RNN inference: update and pass forward

The
state: article

robot
state: entity

picked
state: action

up
state: phrase

Elegant idea: every new token edits the running summary.

Why RNNs are limited

Sequential

Step t waits for $t - 1$.

Bottleneck

Everything squeezes into one state.

Long paths

Distant tokens interact through many steps.

Good concept, hard scaling story.

Modern recurrence revisits the idea

Structured state updates can be faster and stabler:

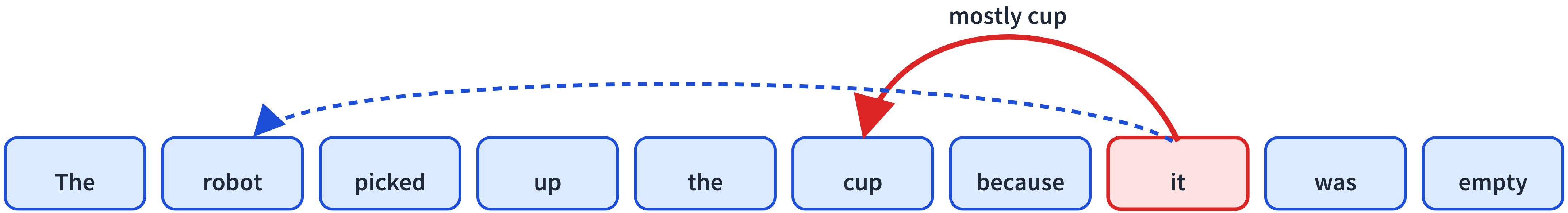
$$\text{state}_t = A\,\text{state}_{t-1} + Bx_t$$

> State-space models and linear RNNs keep the recurrence idea, but redesign it for modern hardware.

Transformers dominate today, but recurrence is not dead.

Different idea: let tokens look directly

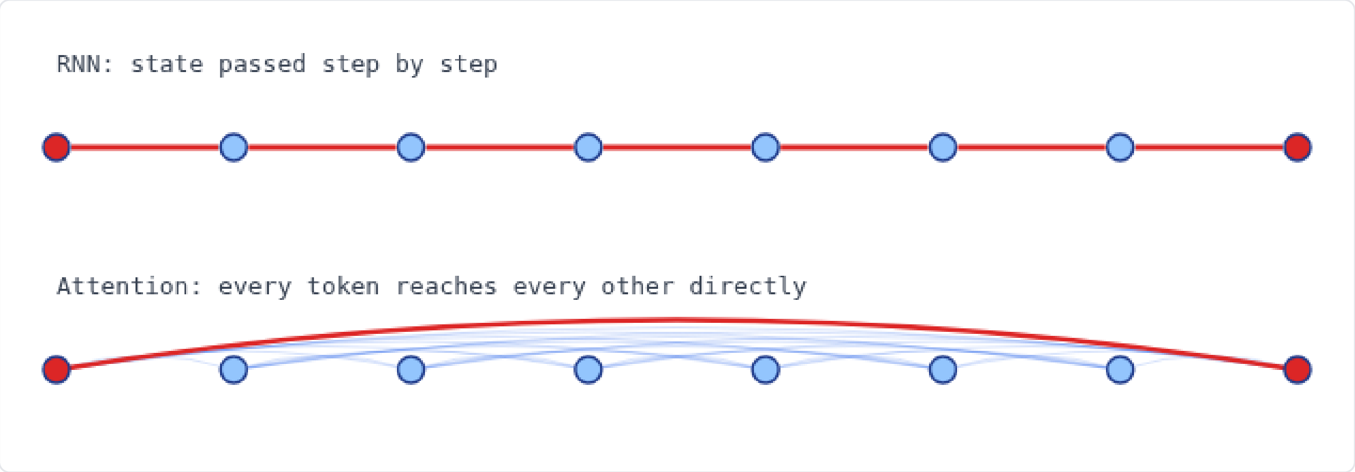
Instead of compressing the past, let a token retrieve the information it needs.



Reaching across the sequence LIVE

Drag the sequence length: the RNN path grows, the attention path stays one hop.

sequence length n 8



RNN: first ↔ last token is **7 steps** apart (must go one at a time)

Attention: any pair connects in **1 hop**, at the cost of **64** pairwise scores (n^2)

Longer sequences stretch the RNN path but never the attention path — that is why attention parallelizes and captures long-range links, at quadratic cost.

Attention is content-based lookup

Query

What am I looking for?

Key

What do I advertise?

Value

What information do I provide?

A query matches keys, then reads a weighted mix of values.

Queries, keys, values are learned projections

For token embedding $\mathbf{x}_i$:

query

$\mathbf{q}_i = W_Q \mathbf{x}_i$

key

$\mathbf{k}_i = W_K \mathbf{x}_i$

value

$\mathbf{v}_i = W_V \mathbf{x}_i$

Same embedding, three roles.

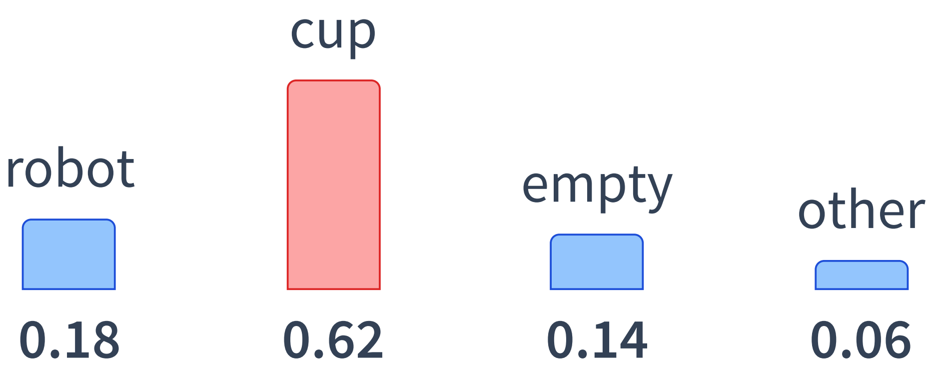
Score: which token matters?

For query token **it**:

$$\mathrm{score}(i, j) = \mathbf{q}_i^\top \mathbf{k}_j$$

candidate key	score
robot	medium
cup	**high**
empty	medium
the	low

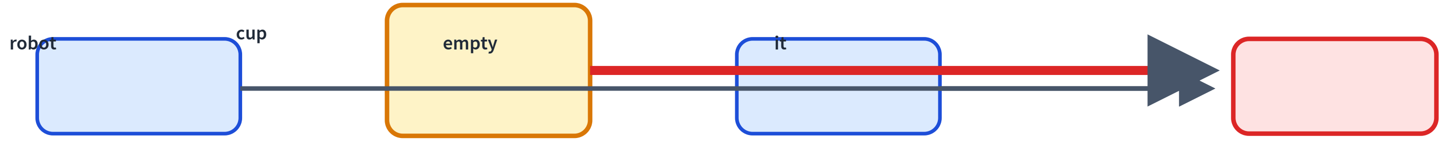
Softmax turns scores into weights



Attention weights are a probability distribution over tokens.

Then mix the values

$$\text{context}_i = \sum_j \alpha_{ij} \mathbf{v}_j$$



The new vector for **it** carries information from **cup**.

Scaled dot-product attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

$QK^\top$

all pairwise relevance scores

V

the information being mixed

The $\sqrt{d_k}$ scaling keeps softmax numerically well-behaved.

Scores → weights LIVE

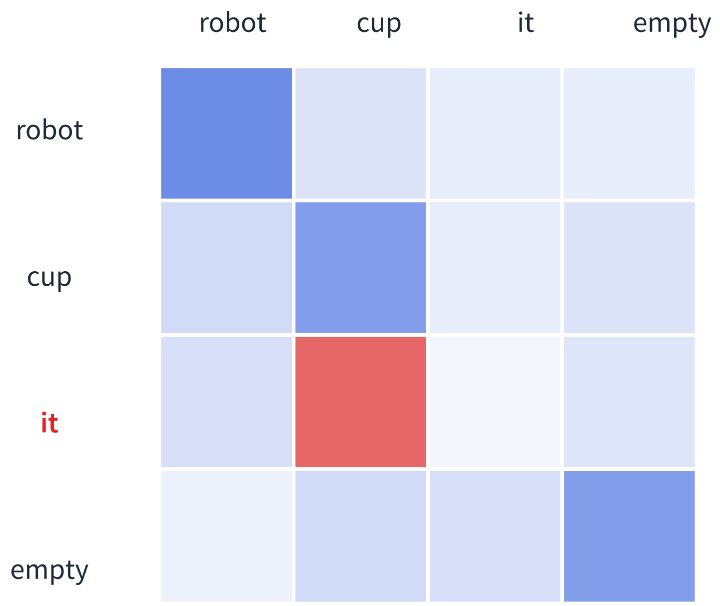
Softmax turns raw scores into attention weights — scaling sharpens or flattens them.

temperature 1.0

dog 1.0 cat 3.0 bird 1.5 fish 0.5

Softmax turns scores into probabilities. Low temperature sharpens; high temperature flattens (the decoding knob in L20/L21).

Attention can be visualized as a heatmap



Rows ask: where does this token attend?

The row for **it** focuses on **cup**.

Caveat: attention is inspectable, but not always a faithful explanation.

Real attention in a transformer LIVE MODEL

Real BERT attention — pick a head, click a query word, read where it looks.

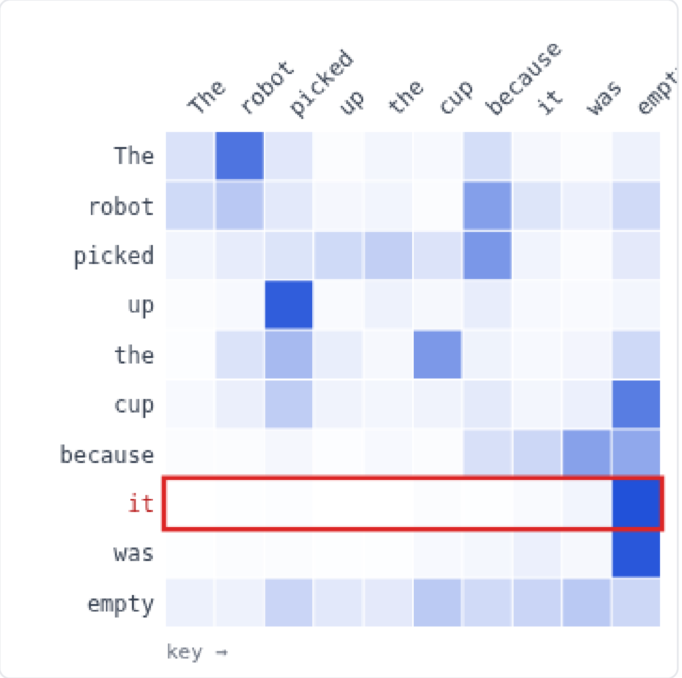
"The robot picked up the cup because it was empty"

"The chef tasted the soup because it was bland"

head: it → empty

head: previous token

head: broad context



query: it attends to…

empty	0.97
was	0.02
it	0.01
cup	0.00
picked	0.00
robot	0.00

Multi-head attention

syntax

subject ↔ verb

coreference

it ↔ cup

local

nearby phrases

Different heads can retrieve different kinds of information in parallel.

Attention still needs position

Attention sees a set of vectors unless we tell it where each token is.

```
x = token_embedding(input_ids)
x = x + position_embedding(positions)
```

Modern note: many LLMs use relative or rotary position information (RoPE), not just learned absolute positions.

Position tells the model that “dog bites man” differs from “man bites dog.”

Does word order matter? LIVE

Toggle positions: without them, attention treats the sentence as a bag of words.

Same three words, two orders. Watch the attention for **bites** with and without position information.

add positions: OFF

“dog bites man” — query **bites** attends to:

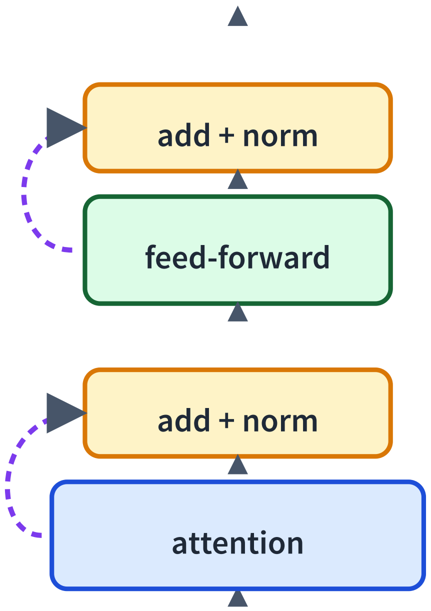
dog	0.50
bites	0.00
man	0.50

“man bites dog” — query **bites** attends to:

man	0.50
bites	0.00
dog	0.50

Identical — without positions the model sees a **set**, so it cannot tell subject from object.

The transformer block



Attention: tokens communicate.

Feed-forward: each token thinks locally.

Residuals and normalization
keep deep stacks trainable.

Two jobs inside each block

attention

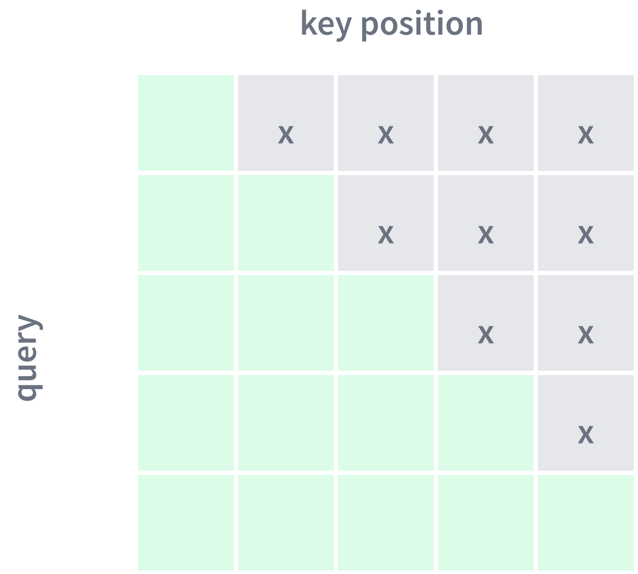
talk across tokens

MLP

think inside each token

Stack this block many times and representations become richer.

Causal masking prevents peeking



A language model predicts the future.

So token t can only attend to positions $\leq t$.

Future scores are set to $-\infty$ before softmax.

Causal masking, interactively LIVE

Toggle the mask and pick a query word — watch the weights renormalize over allowed positions.

A decoder predicts the next word, so a token may only attend to itself and earlier tokens.

causal mask: ON

query: on (position 4)

The	0.13
cat	0.40
sat	0.12
on	0.34
the	–
mat	–

Future words are blocked, so the weights renormalize over positions ≤ t.

Encoder vs decoder

Encoder

Every token can look both directions. Good for understanding.

Decoder

Tokens look left only. Good for generation.

Qwen/GPT-style LLMs are decoder-only causal transformers.

Why transformers took over

Parallel

All positions at once during training.

Long-range

Any token reaches any other in one hop.

Scalable

Bigger models and data kept improving.

Now we have the architecture

Next we need the training task:

predict the next token at massive scale

L20: pretraining language models from first principles.

Recap

✓ RNNs summarize the past with a **hidden state**.

✓ Attention retrieves information directly between **tokens**.

✓ Queries, keys, and values implement learned retrieval.

✓ Transformer blocks stack **attention** and **feed-forward** computation.

✓ Causal masks turn transformers into language generators.